# 위상수학적 음악 분석 — Step 2

## 두 가지 음악 생성 알고리즘

본 장에서는 본 연구의 두 가지 음악 생성 알고리즘 — Algorithm 1 (확률적 샘플링) 과 Algorithm 2 (신경망 기반 시퀀스 모델) — 의 핵심 아이디어와 설계 의도를 설명한다.

### 표기 정의

본 장에서 사용할 표기를 다음과 같이 통일한다.

| 기호 | 의미 | hibari 값 |
|---|---|---|
| $T$ | 시간축 길이 (8분음표 단위) | $1{,}088$ |
| $N$ | 고유 note 수 ((pitch, duration) 쌍) | $23$ |
| $C$ | 발견된 전체 cycle 수 | 최대 $48$ |
| $K$ | 선택된 cycle subset 크기 ( $K \leq C$ ) | $\{10, 17, 48\}$ |
| $O$ | 중첩행렬, $O \in \{0,1\}^{T \times K}$ | — |
| $L_t$ | 시점 $t$ 에서 추출할 note 개수 | $\sum_t L_t \approx 165$ |
| $V(c)$ | cycle $c$ 의 vertex (note label) 집합 | $ |
| $R$ | 재샘플링 최대 시도 횟수 | $50$ |
| $B$ | 학습 미니배치 크기 | $32$ |
| $E$ | 학습 epoch 수 | $200$ |
| $H$ | DL 모델의 hidden dimension | $128$ |

---

## 2.10 Algorithm 1 — 확률적 샘플링 기반 음악 생성

### 알고리즘 개요

Algorithm 1은 **중첩행렬의 ON/OFF 패턴**을 직접 참조하여, 각 시점에서 활성화된 cycle들이 공통으로 포함하는 note pool로부터 확률적으로 음을 추출하는 규칙 기반 알고리즘이다. 신경망 학습 없이 즉시 생성이 가능하며, 중첩행렬이 곧 "구조적 seed" 역할을 한다.

### 핵심 아이디어 (3가지 규칙)

**규칙 1. 시점 $t$ 에서 활성 cycle이 있는 경우**, 즉

$$\sum_{c=1}^{K} O[t, c] > 0$$

일 때, 활성화되어 있는 모든 cycle들의 vertex 집합의 **교집합**

$$I(t) = \bigcap_{c \,:\, O[t,c]=1} V(c)$$

에서 note 하나를 **균등 추출**한다. 여기서 "균등 추출"이란 집합 $I(t)$의 모든 원소가 동일한 확률 $1/|I(t)|$로 선택된다는 의미이다 (이산균등분포). 만약 교집합이 공집합이면 ( $I(t) = \emptyset$), 전체 note pool $P$ 에서 균등 추출한다. 이 규칙은 "여러 cycle이 동시에 살아 있을 때, 그 cycle들이 모두 공유하는 note는 음악적으로도 가장 핵심적인 음"이라는 가정을 반영한다.

**규칙 2. 시점 $t$ 에서 활성 cycle이 없는 경우**, 즉

$$\sum_{c=1}^{K} O[t,c] = 0$$

일 때, 인접 시점 $t-1, t+1$에서 활성화된 cycle들의 vertex의 **합집합**

$$A(t) = \bigcup_{c\,:\,O[t-1,c]=1} V(c) \cup \bigcup_{c\,:\,O[t+1,c]=1} V(c)$$

을 계산한 뒤, 전체 note pool에서 이 합집합을 제외한 영역 $P \setminus A(t)$ 에서 균등 추출한다.

이 규칙은 다음과 같은 의미를 가진다. 활성 cycle이 없는 시점에서도 음악은 흘러가야 하므로 음을 하나 골라야 하는데, 만약 인접 시점의 cycle 멤버 노트를 그대로 골라 버리면, 청자가 들었을 때 마치 그 cycle이 시점 $t$에도 살아 있는 것처럼 들리게 된다. 즉, 원래 분석상으로는 죽어 있어야 할 위상 구조가 인위적으로 살아있는 것처럼 번지는 현상이 생긴다. 이를 막기 위해 인접 cycle의 vertex를 의도적으로 회피하여, "활성 cycle 없음"이라는 정보가 청각적으로도 그대로 보존되도록 한다.

**규칙 3. 중복 onset 방지.** 같은 시점 $t$에서 동일한 (pitch, duration) 쌍이 두 번 추출되지 않도록 `onset_checker`로 검사하며, 충돌이 발생하면 최대 $R$회까지 재샘플링한다. $R$회 모두 실패하면 그 시점의 해당 note 자리는 비워둔다.

### 출력

알고리즘은 (start, pitch, end) 형태의 음표 리스트 $G$를 출력하며, 이를 MusicXML로 직렬화하면 곧바로 악보 및 오디오로 재생할 수 있다.

---

## 2.11 Algorithm 2 — 신경망 기반 시퀀스 음악 생성

### 알고리즘 개요

Algorithm 2는 중첩행렬을 입력, 원곡의 multi-hot note 행렬을 정답 레이블로 두고 매핑

$$f_\theta : \{0,1\}^{T\times C} \longrightarrow \mathbb{R}^{T\times N}$$

을 학습한다 (FC 모델은 시점별 독립이므로 $\{0,1\}^C \to \mathbb{R}^N$). 학습된 모델은 학습 시 보지 못한 cycle subset이나 노이즈가 섞인 중첩행렬에서도 위상 구조를 보존하는 음악을 복원할 수 있다는 점에서, Algorithm 1의 단순 반복 생성과 차별화된다.

### 모델 아키텍처 비교

본 연구는 동일한 학습 파이프라인 위에서 세 가지 모델 아키텍처를 비교한다.

| 모델 | 입력 형태 | 시간 정보 처리 방식 | 파라미터 수 (대략) |
|---|---|---|---|
| FC | $(B, C)$ | 시점별 독립 (시간 무시) | $\approx 4\times 10^4$ |
| LSTM (2-layer) | $(B, T, C)$ | 순차 hidden state | $\approx 2\times 10^5$ |
| Transformer (2-layer, 4-head) | $(B, T, C)$ | self-attention | $\approx 4\times 10^5$ |

FC는 가장 단순하며 각 시점의 위상 정보만 본다. LSTM은 hidden state를 통해 과거 정보를 누적적으로 본다. Transformer는 self-attention으로 모든 시점이 모든 시점을 본다. 세 모델은 곧 "위상 정보를 음악으로 옮길 때 시간 문맥을 얼마나 활용해야 하는가"라는 질문에 대한 세 가지 답이다.

### 학습 데이터 구성과 증강

원본 학습 쌍은 $X \in \{0,1\}^{T \times C}$, $y \in \{0,1\}^{T \times N}$이다. 여기서 $X$는 중첩행렬이고 $y$는 같은 시간축의 multi-hot note 행렬(시점 $t$에 활성인 note를 1로 표시)이다. 본 연구는 세 가지 증강 전략을 적용하여 학습 데이터를 약 $7 \sim 10$배 늘린다.

**(1) Subset Augmentation.** $K \in \{10, 15, 20, 30\}$의 cycle subset에 대한 중첩행렬을 생성하여, 동일한 정답 $y$에 매핑한다. 이를 통해 모델은 "불완전한 위상 정보로부터도 원곡을 복원하는" 강건한 표현을 학습한다.

**(2) Circular Shift.** 시간축을 회전하는 증강이며, ** $X$와 $y$를 동일한 양만큼 함께 회전**한다. 즉

$$X' = \mathrm{roll}(X, s, \mathrm{axis} = 0), \qquad y' = \mathrm{roll}(y, s, \mathrm{axis} = 0)$$

로 처리한다 (여기서 $s$는 같은 난수). 이렇게 하면 시점 $t$의 위상 정보와 시점 $t$의 note가 시점 $t+s$로 똑같이 옮겨가므로, 모델이 학습해야 할 매핑 자체는 **변하지 않은 채** 시작점만 달라진다. 만약 $X$에만 회전을 적용하면 $X$와 $y$의 시간축이 어긋나 학습 데이터가 망가지므로, 두 행렬을 항상 함께 회전해야 한다.

**(3) Noise Injection.** $X$에 확률 $p = 0.03$로 bit flip을 가한다 ($y$는 그대로). overfitting을 막고 정규화 효과를 얻기 위함이다.

### 학습 손실 함수

각 시점에서 여러 note가 동시에 활성화될 수 있으므로 (multi-label 문제), 2.8절에서 정의한 binary cross-entropy 손실을 사용한다. PyTorch의 `BCEWithLogitsLoss`는 sigmoid를 손실 안에 포함시킨 수치 안정 버전이며, 양성/음성 클래스의 불균형을 보정하기 위해 `pos_weight` 파라미터에 (#negatives / #positives) 비율을 넣는다.

### 추론 단계

학습이 끝난 모델 $f_{\theta^*}$로 음악을 생성할 때는 다음 순서를 따른다.

1. 중첩행렬 $O_{\text{test}}$를 모델에 통과시켜 logit $\hat{z}$ 획득
2. sigmoid 적용: $P = \sigma(\hat{z}) \in [0,1]^{T \times N}$
3. **Adaptive threshold** (2.8절): $\theta = \mathrm{quantile}(P, 1 - 0.15)$로 임계값 결정
4. $P[t,n] \geq \theta$인 모든 $(t,n)$ 쌍에 대해 note $n$을 시점 $t$에 활성화
5. 필요 시 onset gap 제약 적용 (인접 시점 onset 너무 잦으면 건너뜀)

이 결과로 $G = [(start, pitch, end), \ldots]$ 음표 리스트를 얻는다.

---

## 2.12 두 알고리즘의 비교 요약

| 항목 | Algorithm 1 (Sampling) | Algorithm 2 (DL) |
|---|---|---|
| 학습 필요 여부 | 불필요 (즉시 생성) | 필요 ($E$ epoch) |
| 결정성 | 확률적 (난수) | 학습 후 결정적 |
| 일반화 | 같은 곡 내부에서만 | 보지 못한 cycle subset도 생성 |
| 위상 보존 방식 | 교집합 규칙으로 직접 강제 | 손실함수를 통해 간접 |
| hibari 실측 (생성 1회) | $\sim 50\mathrm{ms}$ | $\sim 100\mathrm{ms}$ (학습 후) |
| hibari 실측 (학습 포함) | — | $30\mathrm{s} \sim 3\mathrm{min}$ |

**해석.** Algorithm 1은 위상 정보를 직접 규칙으로 강제하므로 cycle 보존도 측면에서 가장 신뢰할 수 있는 기준선 역할을 한다. 반면 Algorithm 2는 학습된 잠재 표현을 통해 부드러운 생성이 가능하며, 학습 데이터에 없는 cycle subset에 대해서도 합리적인 음악을 만들어낸다. 본 연구의 실험에서는 두 알고리즘이 상호 보완적임을 보였다 — Algorithm 1은 위상 보존도에서, Algorithm 2는 음악적 자연스러움에서 각각 우위를 보인다 (Step 4 실험 결과 참조).

---

## 참고문헌 (Step 2 추가분)


- Hochreiter, S., & Schmidhuber, J. (1997). "Long Short-Term Memory". Neural Computation, 9(8), 1735-1780.
- Vaswani, A., et al. (2017). "Attention is all you need". NeurIPS.
- Kingma, D. P., & Ba, J. (2015). "Adam: A method for stochastic optimization". ICLR.